

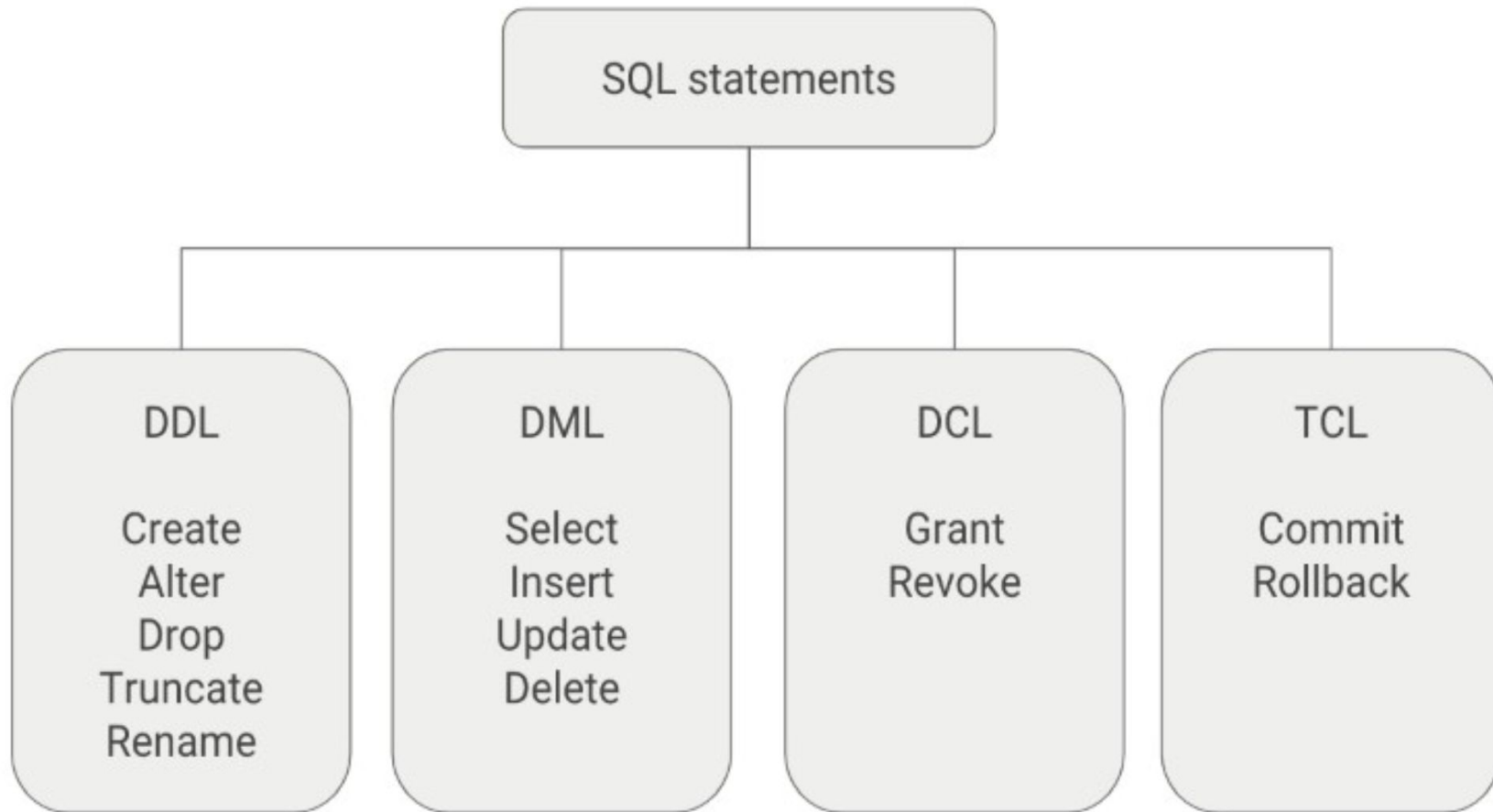
Основные объекты Postgres. Основы DDL и DML

Лекция 2

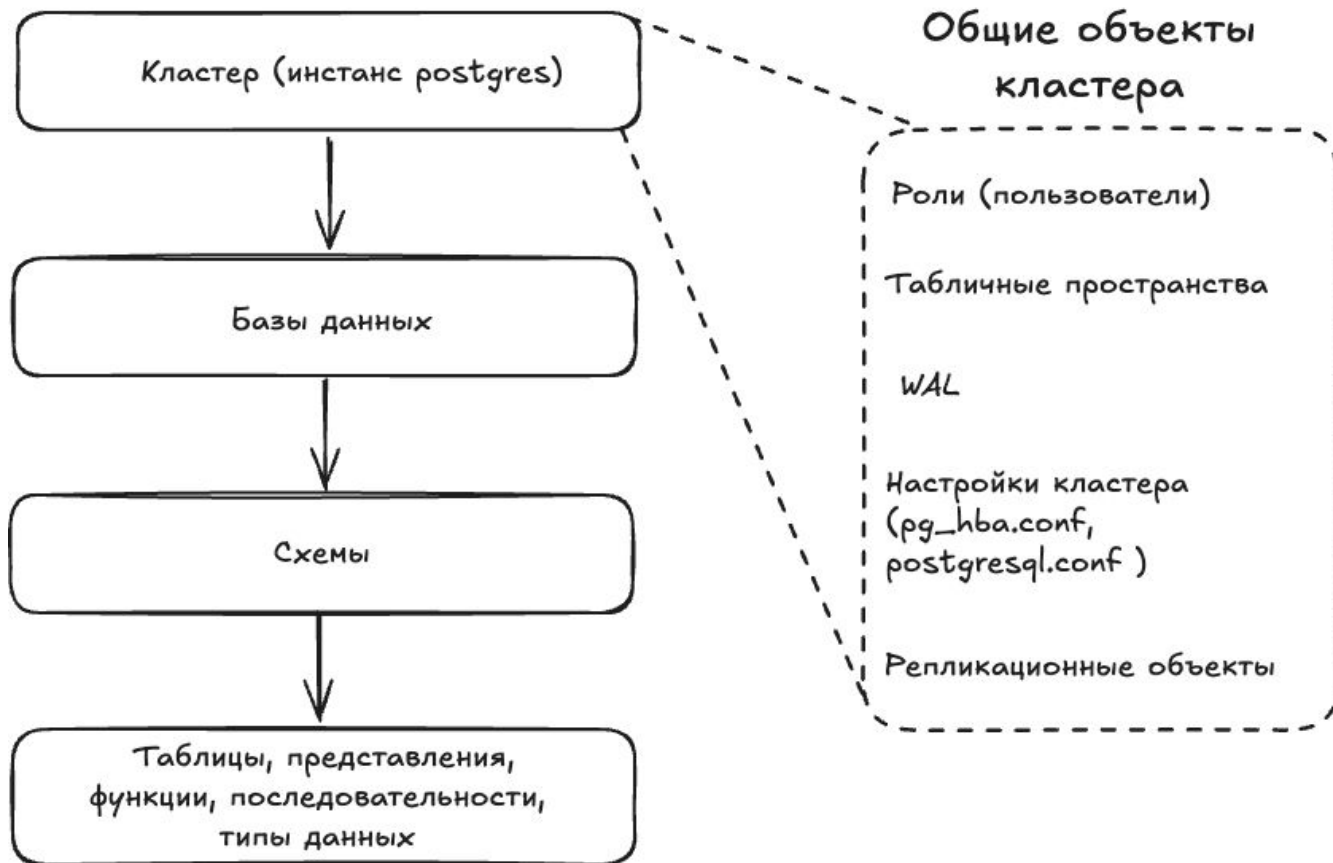
План

- Организация данных в Postgres
- Основы DDL (Data Definition Language) – язык определения данных
- Основы DML (Data Manipulation Language) – язык манипулирования данными
- Основные типы данных Postgres

Классификация SQL



Иерархия объектов Postgres



Базы данных в Postgres

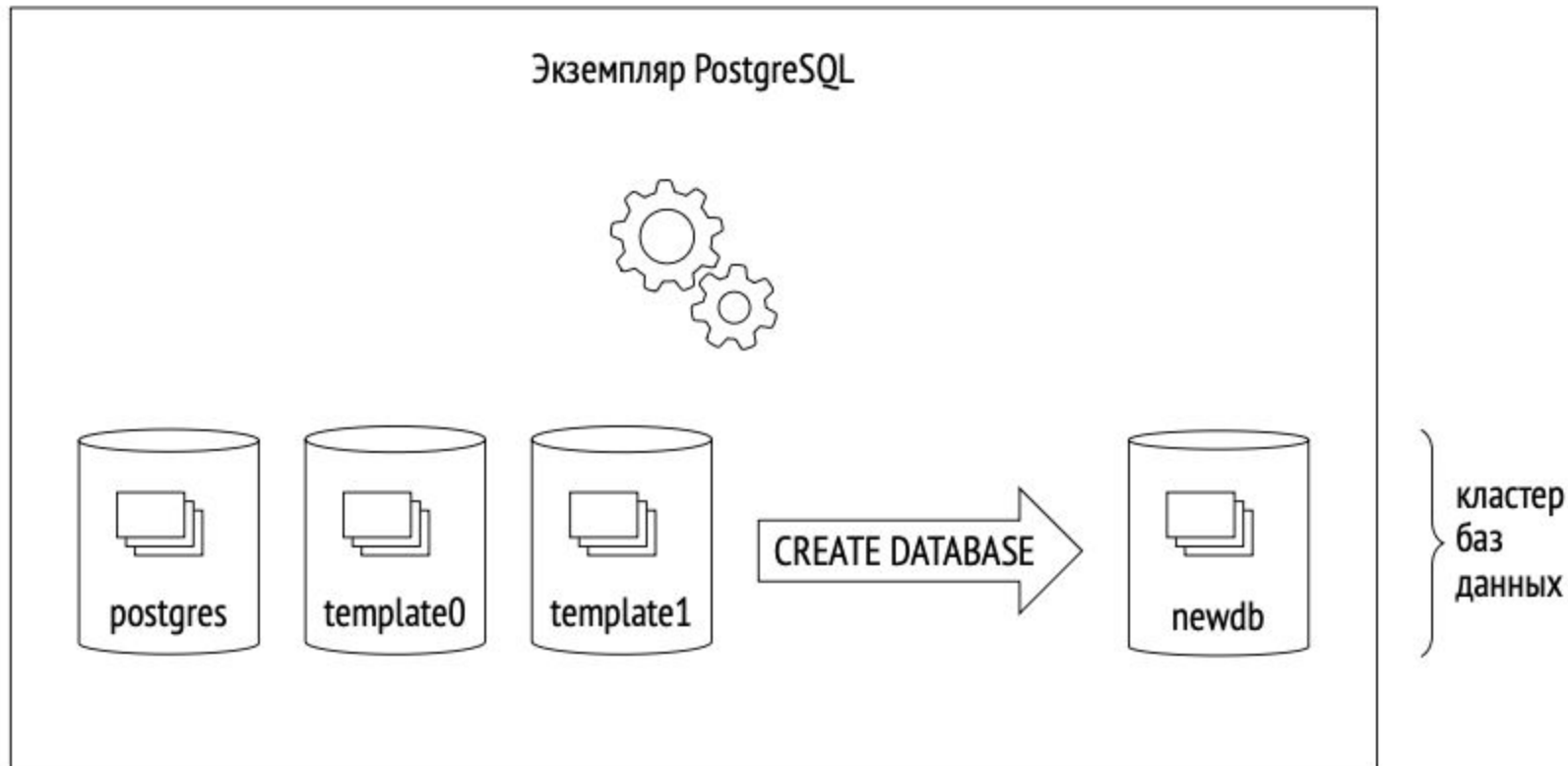
База данных – это логически изолированный набор объектов для хранения и управления данными.

- Помимо пользовательских объектов, каждая база данных имеет свой системный каталог – набор специальных объектов, у которых имя начинается с *pg_**
- Работа с Postgres осуществляется **только при подключении к конкретной базе данных**

Создание кластера Postgres

- Происходит инициализация каталога PGDATA (имя переменной окружения). В нем хранятся все данные кластера.
- При инициализации PGDATA в кластере создаются 3 базы данных:
 - **template0** – немодифицируемая “эталонная” база данных. Используется для восстановления баз данных из логической резервной копии (дампа) или при создании базы данных с кодировкой, отличной от template1
 - **template1** – служит шаблоном для всех остальных баз данных, которые пользователь может создать в кластере
 - **postgres** – обычная база данных, которую можно использовать по своему усмотрению

Создание кластера Postgres



Создание и удаление базы данных

CREATE DATABASE libdb; – создаст базу libdb, копирует template1

CREATE DATABASE libdb **TEMPLATE** template0 **ENCODING** 'SQL_ASCII'; –
меняем кодировку, поэтому копируем template0

<https://postgrespro.ru/docs/postgresql/18/sql-createdatabase>

Каждая база данных имеет свою копию **системного каталога** – набора специальных таблиц с префиксом *pg_* для хранения метаданных.

<https://postgrespro.ru/docs/postgresql/18/catalogs>. Пример:

SELECT * FROM pg_tables; – выведет список таблиц в базе данных

Удаление базы данных:

DROP DATABASE libdb; – удаляем базу данных

<https://www.postgresql.org/docs/18/sql-dropdatabase.html>

Схемы

Схема – это логическое пространство имен внутри базы данных, которое используется для группировки и организации объектов базы данных.

- Внутри схемы два объекта одного типа не могут иметь одно имя.
- Таблицы, последовательности, индексы, представления, материализованные представления и внешние таблицы существуют в одном пространстве имён. Это означает, например, что таблица и индекс внутри схемы также не могут иметь одинаковые имена.
- Таблицы и другие объекты могут иметь одинаковые имена, если они определены в разных схемах

Зачем нужны схемы?

- Чтобы в одной базе сосуществовали разные приложения, и при этом не возникало конфликтов имен.
- Чтобы одну базу данных могли использовать несколько пользователей, независимо друг от друга – схемы позволяют гранулярно настроить доступ для разных пользователей. По-умолчанию, только суперпользователь и пользователь, который владеет схемой, может создавать объекты в схеме.
- Чтобы объединить объекты базы данных в логические группы для облегчения управления ими.

Список служебных схем

- **public** – используется по умолчанию, когда явно не указана другая схема при обращении к таблице или другому объекту
- **pg_catalog** – схема системного каталога
- **information_schema** – альтернативное представление pg_catalog, совместимое со стандартом SQL
- **pg_toast** – используется для объектов TOAST (The Oversized-Attribute Storage Technique)
- **pg_temp** – схема для временных таблиц

Табличные пространства

Табличные пространства (ТП) – определяют физическое расположение данных

- **pg_default** – ТП по-умолчанию для всех новых данных, располагается в PGDATA/base
- **pg_global** – ТП для данных системного каталога, общих для всего кластера. Распологается в PGDATA/global

ТП позволяют разделить логическую структуру и физическое хранение: одно ТП может использоваться разными базами данных, и одна база данных может размещаться в разных ТП.

Пользовательское ТП – это произвольный каталог. В PGDATA помещается лишь символьная ссылка на него.

Основы DDL и DML

Создание и удаление схемы

CREATE SCHEMA library; – создает схему library, которой владеет текущий пользователь

CREATE SCHEMA library AUTHORIZATION lib_user; – создает схему, которой владеет lib_user;

CREATE SCHEMA IF NOT EXISTS library; – не вернет ошибку, если схема уже существует

DROP SCHEMA library; – удаляет пустую схему

DROP SCHEMA library CASCADE; – удаляет схему, содержащую объекты

<https://postgrespro.ru/docs/postgrespro/current/sql-createschema>

<https://postgrespro.ru/docs/postgresql/18/sql-dropschema>

Как создать пользователя для владения схемой?

В postgres используется концепция ролей. Роль – это пользователь или группа пользователей, зависит от настройки роли (детали в лекции 4).

`CREATE ROLE lib_user WITH LOGIN PASSWORD 'pass';` – создаст пользователя `lib_user` с паролем `pass`

`GRANT CREATE ON DATABASE libdb TO lib_user;` – даст пользователю `lib_user` право на создание схем

`DROP ROLE lib_user;` – удаляет пользователя, только если все его привилегии удалены, либо переданы другому пользователю

`REASSIGN OWNED BY lib_user to another_user;` – передает привилегии

`DROP OWNED BY lib_user;` – удаляет привилегии пользователя

Создание таблиц

Упрощенная форма запроса для создания таблиц выглядит так

```
CREATE TABLE схема.имя-таблицы(  
    имя-поля тип-данных [ограничения-целостности],  
    ...  
    имя-поля тип-данных [ограничения-целостности],  
    [ограничения-целостности],  
    [первичный-ключ],  
    [внешний-ключ]  
);
```


Создание таблиц (пример)

readers						
card_no	passport_sn	passport_no	first_name	last_name	birth_date	registered_at
2023-234	2050	122124	Victor	Ivanov	1993-04-05	2023-05-03

```
CREATE TABLE library.readers (  
    card_no VARCHAR(20),  
    passport_sn VARCHAR(4) NOT NULL,  
    passport_no VARCHAR(6) NOT NULL,  
    first_name TEXT NOT NULL,  
    last_name TEXT NOT NULL,  
    birth_date DATE NOT NULL CHECK (birth_date < CURRENT_DATE),  
    registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    UNIQUE (passport_sn, passport_no),  
    PRIMARY KEY (card_no)  
);
```

Создание таблиц (пример)

readers							
id	card_no	passport_sn	passport_no	first_name	last_name	birth_date	registered_at
5	2023-234	20 50	122 124	Victor	Ivanov	1993-04-05	2023-05-03

```
CREATE TABLE library.readers (  
    id serial PRIMARY KEY,  
    card_no VARCHAR(20) UNIQUE,  
    passport_sn VARCHAR(4) NOT NULL,  
    passport_no VARCHAR(6) NOT NULL,  
    first_name TEXT NOT NULL,  
    last_name TEXT NOT NULL,  
    birth_date DATE NOT NULL CHECK (birth_date < CURRENT_DATE),  
    registered_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    UNIQUE (passport_sn, passport_no)  
);
```

Удаление таблиц

DROP TABLE `library.readers`; – удаляет таблицу, если нет ограничений
ссылочной целостности (подробнее в лекции 3)

Search path

А если мы не хотим постоянно писать “library.table_name”, а хотим писать “table_name”?

SHOW search_path; выведет -> "\$user", public – это порядок поиска объекта, при отсутствии явного указания схемы

SET search_path TO library,public; – установит search_path для текущего сеанса, теперь **DROP TABLE readers;** будет искать таблицу readers сначала в схеме library, затем в public

SET search_path TO library; – только объекты в схеме library не будут требовать указания схемы

RESET search_path; – сбросит значение до значения по умолчанию

Альтернативные способы задания search_path

- На уровне кластера в postgresql.conf
- На уровне бд
 - `ALTER DATABASE libdb SET search_path TO library;`
 - `ALTER DATABASE libdb RESET search_path;` – сброс
- На уровне роли
 - `ALTER ROLE lib_user SET search_path TO library;`
 - `ALTER ROLE lib_user RESET search_path;` – сброс

<https://postgrespro.ru/docs/postgresql/18/runtime-config-client#GUC-SEARCH-PATH>

<https://postgrespro.ru/docs/postgrespro/current/ddl-schemas>

Вставка данных в таблицу

Упрощенная форма запроса для вставки данных в таблицу выглядит так

INSERT INTO имя-таблицы

[(имя-атрибута, имя-атрибута, ...)]

VALUES (значение-атрибута, значение-атрибута, ...);

Вставка данных в таблицу (пример)

Вставка одной строки:

```
INSERT INTO readers
(card_no, passport_sn, passport_no, first_name, last_name, birth_date)
VALUES ('2026-434', '0304', '123456', 'Victor', 'Egorov', '1994-04-05');
```

Вставка нескольких строк:

```
INSERT INTO readers
(card_no, passport_sn, passport_no, first_name, last_name, birth_date)
VALUES
    ('2026-434', '0304', '123456', 'Victor', 'Egorov', '1994-04-05'),
    ('2026-435', '0305', '123457', 'Vasiliy', 'Starov', '1994-04-05')
RETURNING *;
```



Вернет все вставленные строки в результате

Чтение данных из таблицы

Упрощенная форма запроса для чтения данных из таблицы выглядит так

```
SELECT имя-атрибута, имя-атрибута, ...  
      FROM имя-таблицы  
      [WHERE условие];
```

Если нужно вывести все атрибуты

```
SELECT *  
      FROM имя-таблицы  
      [WHERE условие];
```


Чтение данных из таблицы (пример)

Вывести все данные по всем читателям:

```
SELECT * FROM readers;
```

Вывести все данные по всем Викторам:

```
SELECT * FROM readers WHERE first_name='Victor';
```

Вывести паспортные данные Виктора Егорова:

```
SELECT passport_sn, passport_no FROM readers  
WHERE first_name='Victor' AND last_name='Egorov';
```

Вывести номер читательского у тех, кто зарегался в 2026 году

```
SELECT card_no FROM readers WHERE registered_at > '2026-01-01' ORDER BY  
registered_at DESC; – order by упорядочивает результаты по возрастанию ASC (по-  
умолчанию) или по убыванию DESC
```

Обновление данных в таблице

Упрощенная форма запроса для чтения данных из таблицы выглядит так

```
UPDATE имя-таблицы  
  SET имя-атрибута1 = значение-атрибута1,  
      ...  
      SET имя-атрибутаN = значение-атрибутаN,  
  [WHERE условие];
```



Это условие нужно в 99% случаях, иначе обновятся все строки

Обновление данных в таблице (пример)

Поменяем дату рождения Виктору Егорову

```
UPDATE readers
  SET birth_date = '1994-04-06'
  WHERE first_name = 'Victor' AND last_name = 'Egorov';
```

Поменяем формат номера читательского билета для всех, кто зарегистрировался с начала с 2026 года, добавим для таких номеров префикс 2026-

```
UPDATE readers
  SET card_no = '2026-' || card_no
  WHERE registered_at > '2026-01-01';
```

Альтернатива

```
SET card_no = CONCAT('2026-', card_no)
```

Удаление данных из таблицы

Упрощенная форма запроса для удаления данных из таблицы выглядит так

DELETE FROM имя-таблицы [WHERE условие];

Это условие нужно всегда



Если нужно удалить все данные из таблицы

TRUNCATE TABLE имя-таблицы;

TRUNCATE TABLE имя-таблицы RESTART IDENTITY; – когда нужно сбросить SERIAL ключ

TRUNCATE работает гораздо быстрее!

Удаление данных из таблицы (пример)

Удалим Виктора Егорова

```
DELETE FROM readers WHERE  
    first_name = 'Victor' and last_name = 'Egorov';
```

Удалим все данные из таблицы

```
TRUNCATE TABLE readers;
```

Основные типы данных

Числовые типы

Имя	Размер	Описание	Диапазон
<code>smallint</code>	2 байта	целое в небольшом диапазоне	-32768 .. +32767
<code>integer</code>	4 байта	типичный выбор для целых чисел	-2147483648 .. +2147483647
<code>bigint</code>	8 байт	целое в большом диапазоне	-9223372036854775808 .. 9223372036854775807
<code>decimal</code>	переменный	вещественное число с указанной точностью	до 131072 цифр до десятичной точки и до 16383 — после
<code>numeric</code>	переменный	вещественное число с указанной точностью	до 131072 цифр до десятичной точки и до 16383 — после
<code>real</code>	4 байта	вещественное число с переменной точностью	точность в пределах 6 десятичных цифр
<code>double precision</code>	8 байт	вещественное число с переменной точностью	точность в пределах 15 десятичных цифр
<code>smallserial</code>	2 байта	небольшое целое с автоувеличением	1 .. 32767
<code>serial</code>	4 байта	целое с автоувеличением	1 .. 2147483647
<code>bigserial</code>	8 байт	большое целое с автоувеличением	1 .. 9223372036854775807

Числовые типы (numeric и decimal)

- Применяется там, где важна точность (например, деньги)
- `numeric(точность, масштаб)`, где масштаб определяет количество десятичных цифр в дробной части, справа от десятичной точки, а точность — общее количество значимых цифр в числе.
23.5141 — точность 6, масштаб 4. Если масштаб 0 — это целое число.
- Если масштаб значения, которое нужно сохранить, превышает объявленный масштаб столбца, произойдет округление. Например, `SELECT 0.123::numeric(3, 2)` — вернет 0.12
- Точность и масштаб определяют максимальный, а не фиксированный размер хранения

Числовые типы (serial)

Serial – это псевдотип.

CREATE TABLE имя-таблицы (имя-столбца serial); тоже самое, что

CREATE SEQUENCE имя-таблицы_имя-столбца_seq;

CREATE TABLE имя-таблицы (
 имя-столбца integer NOT NULL
 DEFAULT nextval('имя-таблицы_имя-столбца_seq')
);

ALTER SEQUENCE имя-таблицы_имя-столбца_seq
OWNED BY имя-таблицы.имя-столбца;

Строковые типы

Имя	Описание
<code>character varying(<i>n</i>)</code> , <code>varchar(<i>n</i>)</code>	строка ограниченной переменной длины
<code>character(<i>n</i>)</code> , <code>char(<i>n</i>)</code> , <code>bpchar(<i>n</i>)</code>	строка фиксированной длины, дополненная пробелами
<code>bpchar</code>	строка неограниченной переменной длины с удалением пробелов
<code>text</code>	строка неограниченной переменной длины

На практике имеет смысл использовать только `text` и `varchar`.

Константы символьных типов заключаются в одинарные кавычки.

```
SELECT 'postgres';
```

Если нужна кавычка, как символ, ее надо удвоить `SELECT 'postg' 'res' ;`,
либо вместо кавычки, можно использовать `$$`

```
SELECT $$postg'res$$;
```

Типы даты и времени

Имя	Размер	Описание	Наименьшее значение	Наибольшее значение	Точность
timestamp [(p)] [without time zone]	8 байт	дата и время (без часового пояса)	4713 до н. э.	294276 н. э.	1 микросекунда
timestamp [(p)] with time zone	8 байт	дата и время (с часовым поясом)	4713 до н. э.	294276 н. э.	1 микросекунда
date	4 байта	дата (без времени суток)	4713 до н. э.	5874897 н. э.	1 день
time [(p)] [without time zone]	8 байт	время суток (без даты)	00:00:00	24:00:00	1 микросекунда
time [(p)] with time zone	12 байт	время дня (без даты), с часовым поясом	00:00:00+1559	24:00:00-1559	1 микросекунда
interval [поля] [(p)]	16 байт	временной интервал	-178000000 лет	178000000 лет	1 микросекунда

параметр **p** определяет, сколько знаков после запятой должно сохраняться в секундах

для типа timestamp with time zone существует сокращенное название timestamptz

<https://postgrespro.ru/docs/postgrespro/current/datatype-datetime>

Ввод даты и времени

В Postgres существует множество способов ввода даты и времени. Но лучше выбрать и запомнить какой-то один. Наиболее универсальным считается формат даты-времени ISO 8601.

- `SELECT '2025-10-03'::date;`
- `SELECT '13:23'::time;`
- `SELECT timestamptz '2026-01-15 23:37:41';`
- `SELECT timestamptz '2026-01-15T23:37+03';` – стандарт ISO диктует использование “T” для разделения даты и времени, postgres понимает оба варианта, но в выводе заменяет “T” на пробел
- `current_date`, `current_time`, `current_timestamp` – функции для получения значений по умолчанию, например `SELECT current_timestamp;`

Часовой пояс

timestamp и timestampz – оба хранят значение временной метки UTC. Разница в том, что при выводе timestampz приводится к часовому поясу пользователя (сеанса).

`SET timezone = 'Europe/Moscow';` – задает часовой пояс

`SELECT timestampz '2026-01-15T23:37+00';` → 2026-01-16 02:37:00+03

`SHOW timezone;` – показать текущий часовой пояс

На практике timestampz используется чаще.

Логический тип (boolean)

- Истинные значения: TRUE, 't', 'true', 'y', 'yes', 'on', '1'
- Ложные значения: FALSE, 'f', 'false', 'n', 'no', 'off', '0'

При запросе можем пропускать явные сравнения

```
SELECT * FROM test WHERE is_valid;
```

```
SELECT * FROM test WHERE NOT is_valid;
```

Массивы

```
CREATE TABLE book_tags (  
    book_id INTEGER PRIMARY KEY REFERENCES books(book_id),  
    tags TEXT[]              -- массив тегов  
);
```

```
INSERT INTO book_tags (book_id, tags)
```

```
VALUES (1, ARRAY ['война', 'любовь', 'Наполеон']);
```

 — добавили запись

```
SELECT book_id, tags[1] as first_tag, tags[2] as second_tag FROM book_tags;
```

 book_id	 first_tag	 second_tag
1	война	любовь

Операции с массивами

Добавление элемента в конец (альтернатива оператор ||)

```
UPDATE book_tags SET tags = array_append(tags, 'эпопея') where book_id = 1;
```

Также можно добавить в начало, используя array_prepend()

Обновление элемента по индексу:

```
UPDATE book_tags SET tags[4] = 'мегаэпопея' where book_id = 1;
```

Удаление элемента:

```
UPDATE book_tags SET tags = array_remove(tags, 'эпопея') where book_id = 1;
```

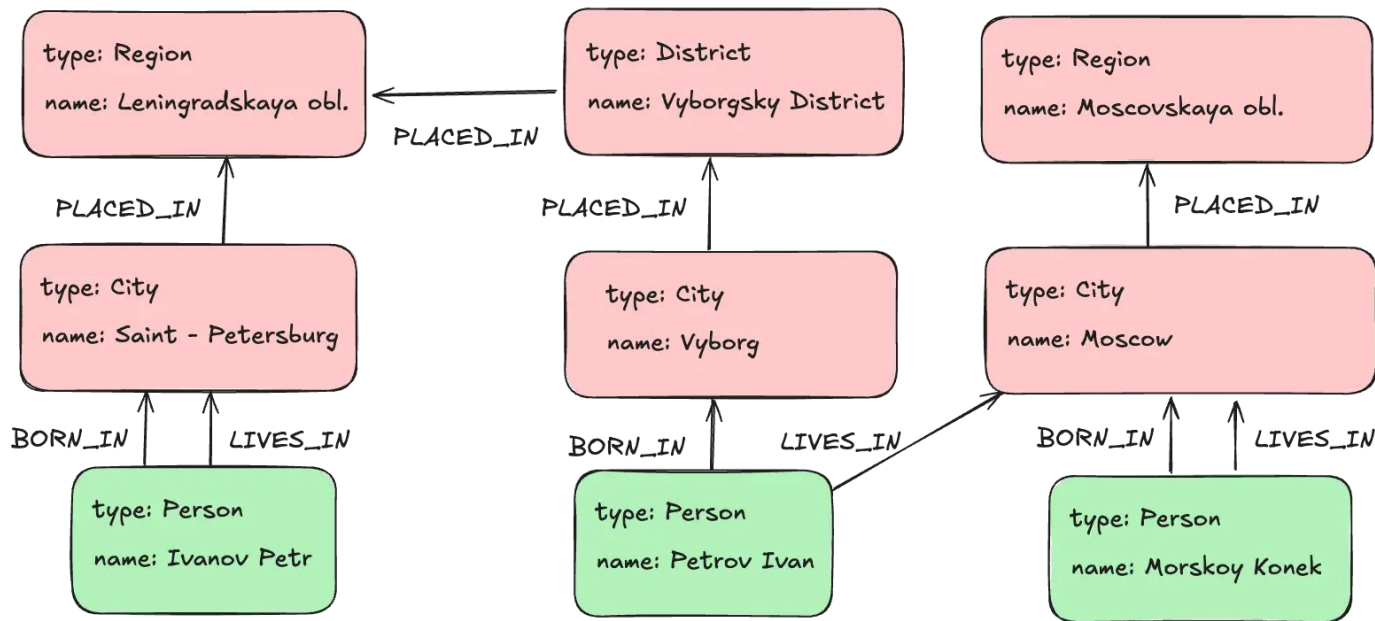
Проверка вхождения массива всех элементов:

```
SELECT book_id FROM book_tags WHERE tags @> ARRAY ['война', 'любовь'];
```

Проверка вхождения хотя бы одного:

```
SELECT book_id FROM book_tags WHERE tags && ARRAY ['детектив', 'любовь'];
```


Бонус: графовые СУБД



MATCH

(PERSON) -[:BORN_IN]-> () -[:WITHIN*0..]-> (lo:Location {name: 'Leningradskaya obl.'})  Petrov Ivan

(PERSON) -[:LIVES_IN]-> () -[:WITHIN*0..]-> (mo:Location {name: 'Moscow'})

RETURN person.name